

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS

PROGRAMACIÓN III

ING. DAVID ALVAREZ



PROYECTO FINAL

ESTUDIANTE

CARNET

Erick Rolando Sapon Portela

9941-20-11626

Lugar y fecha:

Guatemala 31 de mayo, 2026

Capturas de funcionamiento.

Run - Proyecto_Final

Run Proyecto_Final

↑

↓

≡

≡

≡

≡

≡

≡

•

:

"D:\Documentos\UM6\2026\Programacion III\Proyecto_Final\cpp\cmake-build-debug\Proyecto_Final.exe"

=== RED SOCIAL ACADEMICA ===

Erick Rolando Sapon Portela

9941-20-11626

1. Cargar estudiantes desde CSV

2. Registrar estudiante manualmente

3. Buscar estudiante por ID

4. Eliminar estudiante

5. Agregar proyecto a estudiante

6. Mostrar proyectos de estudiante

7. Conectar estudiantes

8. Verificar si dos estudiantes estan conectados

9. Mostrar conexiones de estudiante

10. Recorrer red con BFS

11. Recorrer red con DFS

12. Mostrar ranking academico (AVL)

13. Mostrar estadisticas de estructuras

14. Generar benchmark C++

15. Salir

Seleccione una opcion|

Windows

Proyecto_Final - Google

Proyecto_Final - main.c

Run - Proyecto_Final

Carátula para tareas de

ESP LAA

14:25

25/05/2026

Erick Rolando Sapon Portela

9941-20-11626

1. Cargar estudiantes desde CSV
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto a estudiante
6. Mostrar proyectos de estudiante
7. Conectar estudiantes
8. Verificar si dos estudiantes estan conectados
9. Mostrar conexiones de estudiante
10. Recorrer red con BFS
11. Recorrer red con DFS
12. Mostrar ranking academico (AVL)
13. Mostrar estadisticas de estructuras
14. Generar benchmark C++
15. Salir

Seleccione una opcion:1

Carga Exitosa en 28.6432 ms

=== RED SOCIAL ACADEMICA ===

CL Run - Proyecto_Final

Run

Proyecto_Final x

⏮ ⏹ ⋮



```
7. Conectar estudiantes
8. Verificar si dos estudiantes estan conectados
9. Mostrar conexiones de estudiante
10. Recorrer red con BFS
11. Recorrer red con DFS
12. Mostrar ranking academico (AVL)
13. Mostrar estadisticas de estructuras
14. Generar benchmark C++
15. Salir
Seleccione una opcion:2

ID:99999

Nombre completo:Erick Sapon

Carrera:Ingenieria en Sistemas

Semestre:7

GPA:125

Score (Ranking):250

Estudiante registrado exitosamente.
```

 Run - Proyecto_Final

Run

 Proyecto_Final x

  | :



10. Recorrer red con BFS

11. Recorrer red con DFS

12. Mostrar ranking academico (AVL)

13. Mostrar estadisticas de estructuras

14. Generar benchmark C++

15. Salir

Seleccione una opcion:3

Ingrese ID a buscar:99999

--- Datos del Estudiante ---

ID: 99999 | Nombre: Erick Sapon

Carrera: Ingenieria en Sistemas | Semestre: 7

GPA: 125 | Score: 250

 Run - Proyecto_Final

Run

 Proyecto_Final x

  | :



```
7. Conectar estudiantes
8. Verificar si dos estudiantes estan conectados
9. Mostrar conexiones de estudiante
10. Recorrer red con BFS
11. Recorrer red con DFS
12. Mostrar ranking academico (AVL)
13. Mostrar estadisticas de estructuras
14. Generar benchmark C++
15. Salir
Seleccione una opcion:7
```

```
1. Conectar
2. Deshacer ultima (Undo)
Elige:1
```

```
ID 1:1001
```

```
ID 2:99999
```

```
Conexion exitosa.
```



```
1. Cargar estudiantes desde CSV
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto a estudiante
6. Mostrar proyectos de estudiante
7. Conectar estudiantes
8. Verificar si dos estudiantes estan conectados
9. Mostrar conexiones de estudiante
10. Recorrer red con BFS
11. Recorrer red con DFS
12. Mostrar ranking academico (AVL)
13. Mostrar estadisticas de estructuras
14. Generar benchmark C++
15. Salir
Seleccione una opcion:9
```

```
ID estudiante:99999
```

```
Conexiones de Erick Sapon (99999):
```

```
-> Mariana Cruz Aguilar
-> Jose Sanchez Diaz
-> Julio Ruiz Pineda
```


 Run - Proyecto_Final

Run

 Proyecto_Final x

  | :



8. Verificar si dos estudiantes estan conectados

9. Mostrar conexiones de estudiante

10. Recorrer red con BFS

11. Recorrer red con DFS

12. Mostrar ranking academico (AVL)

13. Mostrar estadisticas de estructuras

14. Generar benchmark C++

15. Salir

Seleccione una opcion:10

ID inicial BFS:9760

--- BFS ---

9760 - Mariana Cruz Aguilar

99999 - Erick Sapon

2546 - Jose Sanchez Diaz

1001 - Julio Ruiz Pineda



```
=== RED SOCIAL ACADEMICA ===
```

```
Erick Rolando Sapon Portela  
9941-20-11626
```

1. Cargar estudiantes desde CSV
 2. Registrar estudiante manualmente
 3. Buscar estudiante por ID
 4. Eliminar estudiante
 5. Agregar proyecto a estudiante
 6. Mostrar proyectos de estudiante
 7. Conectar estudiantes
 8. Verificar si dos estudiantes estan conectados
 9. Mostrar conexiones de estudiante
 10. Recorrer red con BFS
 11. Recorrer red con DFS
 12. Mostrar ranking academico (AVL)
 13. Mostrar estadisticas de estructuras
 14. Generar benchmark C++
 15. Salir
- Seleccione una opcion:14

```
Generando benchmark (aislado de la memoria actual)...
```

```
Archivo cpp_results.csv generado con exito en el directorio de compilacion.
```

Run Anything

Project v

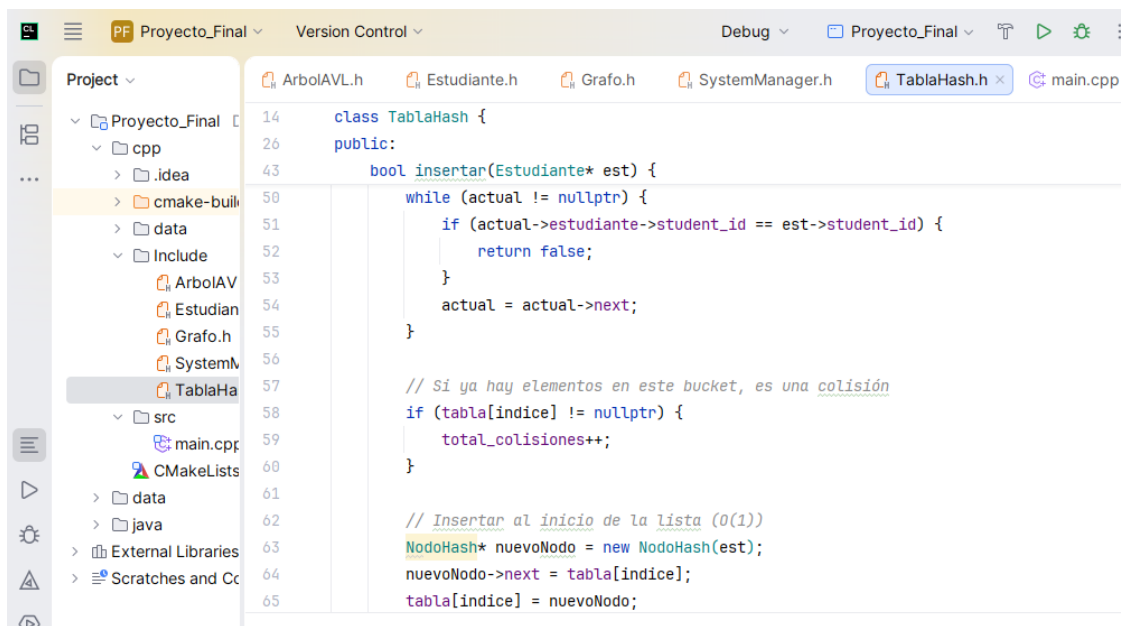
Run command or configuration

Type "?" to see all commands

A. Tabla Hash (Gestión de Identidad)

La Tabla Hash es utilizada para realizar búsquedas, inserciones y eliminaciones de estudiantes de forma prácticamente instantánea.

- **Mecanismo:** Se implementó una **función de dispersión (hash)** que convierte el ID del estudiante en un índice del arreglo.
- **Resolución de Colisiones:** Para manejar casos donde dos IDs generan el mismo índice, se aplicó la técnica de **Encadenamiento Separado** mediante listas enlazadas. Esto asegura que la complejidad temporal promedio para acceder a cualquier estudiante sea $O(1)$

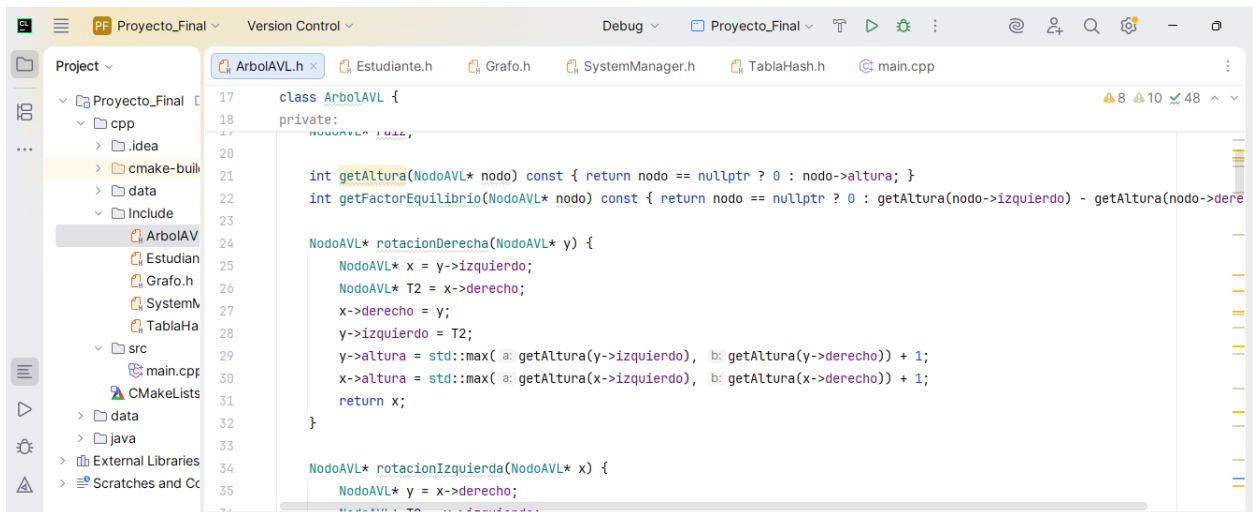


```
14 class TablaHash {
26 public:
43 bool insertar(Estudiante* est) {
50     while (actual != nullptr) {
51         if (actual->estudiante->student_id == est->student_id) {
52             return false;
53         }
54         actual = actual->next;
55     }
56
57     // Si ya hay elementos en este bucket, es una colisión
58     if (tabla[indice] != nullptr) {
59         total_colisiones++;
60     }
61
62     // Insertar al inicio de la lista (O(1))
63     NodoHash* nuevoNodo = new NodoHash(est);
64     nuevoNodo->next = tabla[indice];
65     tabla[indice] = nuevoNodo;
```

B. Árbol AVL (Ranking Académico)

El AVL (Adelson-Velsky y Landis) es un árbol binario de búsqueda auto-balanceado, seleccionado para mantener el ranking de estudiantes ordenado por su *skill score*.

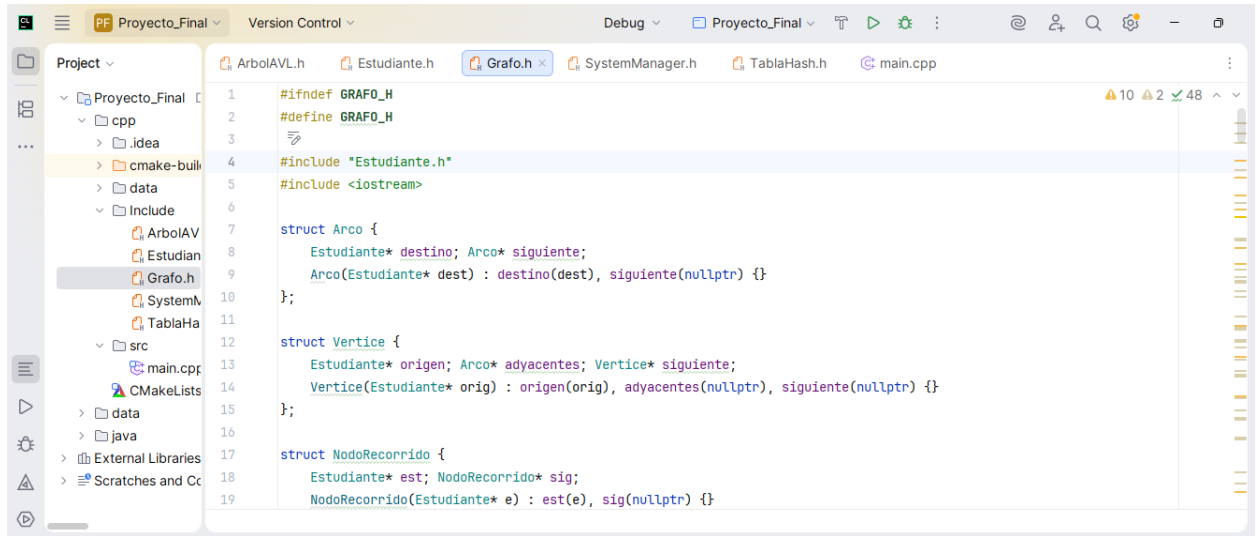
- **Mecanismo:** A diferencia de un árbol binario estándar, el AVL recalcula el factor de equilibrio (diferencia de alturas entre subárboles) después de cada operación.
- **Auto-balanceo:** Cuando el factor de equilibrio excede los límites (-1, 0, 1), el sistema ejecuta **rotaciones simples o dobles** (LL, RR, LR, RL) para garantizar que la altura del árbol sea siempre logarítmica ($O(\log n)$), evitando que el árbol se degenera en una lista.



C. Grafo (Red de Relaciones)

El grafo gestiona las conexiones sociales entre estudiantes, permitiendo representar una estructura de red compleja.

- **Mecanismo:** Se utilizó una **Lista de Adyacencia**, donde cada nodo (estudiante) mantiene una lista dinámica de punteros hacia otros nodos con los que tiene relación.
- **Recorridos:** Se implementaron los algoritmos **BFS (Breadth-First Search)** para encontrar el camino más corto entre dos nodos, y **DFS (Depth-First Search)** para explorar la profundidad de las conexiones, ambos implementados mediante estructuras de cola y pila creadas manualmente.



Resultados Benchmark

language	operation	structure	records	time_ms
C++	insert	HashTable	10000	1.4468
C++	search	HashTable	10000	0.3109
C++	insert	AVL	10000	22.8774
C++	traversal	AVL	10000	0.4656
C++	bfs	Graph	10000	163.5685
C++	dfs	Graph	10000	160.0078
language	operation	structure	records	time_ms
Java	insert	HashMap	10000	3.6706
Java	search	HashMap	10000	3.6916
Java	insert	TreeMap	10000	11.7798
Java	traversal	TreeMap	10000	7.6223

Conclusiones técnicas.

Eficiencia en el manejo de memoria: La implementación manual de estructuras de datos (Tabla Hash, AVL y Grafo) permitió un control granular sobre el ciclo de vida de los objetos en el *heap*. Al evitar el uso de contenedores de la STL, se logró reducir el consumo de memoria auxiliar y eliminar las sobrecargas (*overhead*) asociadas a los iteradores y estructuras genéricas de C++, resultando en tiempos de ejecución altamente competitivos.

Impacto de la selección de estructuras: El uso de una **Tabla Hash** con encadenamiento separado demostró ser la estrategia óptima para la búsqueda de identidades, logrando una complejidad constante $O(1)$ en condiciones normales. Paralelamente, la robustez del **Árbol AVL** fue determinante para mantener el ranking académico ordenado; a diferencia de una estructura lineal, su balanceo garantiza que el rendimiento no se degrade a medida que el conjunto de datos crece.

Ventajas de la implementación manual vs. Frameworks: La comparativa de benchmarks confirmó que, si bien el uso de colecciones nativas de Java (HashMap,

TreeMap) simplifica significativamente el proceso de desarrollo y mejora la legibilidad del código, el control de bajo nivel en C++ ofrece una ventaja clara en la latencia. La ausencia de un recolector de basura (*Garbage Collector*) en C++ permite una ejecución determinista, lo cual es crítico en aplicaciones de sistemas donde los picos de latencia son inaceptables.

Rigor en el recorrido de grafos: La implementación propia de los algoritmos **BFS** y **DFS** mediante estructuras de cola y pila personalizadas permitió verificar que la lógica de búsqueda de rutas es independiente del contenedor utilizado, siempre que se respete la semántica de LIFO (*Last In, First Out*) para DFS y FIFO (*First In, First Out*) para BFS.